Progress, Not Perfection

Version 2 Product and Build Plan

STATUS	DATE
Product plan	**July 26, 2026**
PRODUCT	**ASSISTANT**
Progress, Not Perfection	**D.E.E.D.S.**

Version 2 turns PNP from a capable personal dashboard into a dependable personal operating system for web, iPhone, and Android.

Executive summary

Version 2 turns Progress, Not Perfection from a capable personal dashboard into a dependable personal operating system for web, iPhone, and Android.

The product will continue to be shaped through real daily use, but it will be built for anyone who wants help deciding what matters, protecting their health and relationships, making forward progress, and understanding their own patterns without being judged by a scorecard.

Version 2 has four priorities:

1. Make the existing product stable, coherent, and easy to maintain.
2. Make personal data dependable across devices, including offline use.
3. Make D.E.E.D.S. a useful personal assistant that responds to current PNP data.
4. Deliver true mobile applications with native features and private, on-device intelligence where supported.

The goal is not to add AI everywhere. The goal is to make the product more perceptive, more useful, and more personal without introducing a required per-message AI cost.

1. Product vision

Progress, Not Perfection helps a person run the whole of their life from one calm place.

It combines tasks, goals, health, relationships, business, calendar, mail, journaling, reporting, and personal guidance. It helps the user answer:

- What needs my attention now?
- What can wait?
- What kind of day do I realistically have capacity for?
- Am I moving toward my goals?
- What patterns are helping or hurting me?
- What should I do next?

PNP should feel like a private command center and a thoughtful personal butler, not a productivity contest.

2. Product principles

Progress, not perfection

Trends matter more than isolated days. The product should make partial progress visible and useful.

User judgment comes first

D.E.E.D.S. recommends, explains, and helps the user act. The user makes the decision.

People are not scorecards

Health, relationships, and emotional life must be handled with care. Relationship records describe the partner, friend, parent, or teammate the user chose to be. They are not ratings of another person.

Calm by default

The interface should reduce cognitive load. Important items rise naturally. Secondary information remains available without dominating the screen.

Private by design

Collect only data needed for a user-facing purpose. Make sources, permissions, exports, disconnection, and deletion clear.

Local first, cloud capable

Core functions should work without a connection. Cloud sync should make data available across devices without making the product unusable offline.

Explain the recommendation

Every D.E.E.D.S. suggestion should show the evidence behind it, such as urgency, calendar timing, available energy, goal connection, or an approaching deadline.

No unnecessary save buttons

Routine input should save automatically and confirm quietly.

3. Version 1 baseline

Version 1 has established the product model and the visual direction. It includes:

- Executive, Today, Week, Business, All Tasks, Health, Relationships, Calendar, Mail, Journal, Feedback, Goals, Review, and Settings areas
- editable tasks, custom lists, priorities, recurring maintenance, scheduling, notes, and completion history
- daily physical, mental, emotional, food, medication, hygiene, and activity tracking
- Oura integration and automatic sleep population
- Google Calendar and Gmail integration
- local browser storage, export/import, and Google Drive backup
- KPI and forward-motion reporting
- D.E.E.D.S. recommendations based on structured app data
- responsive web and progressive web app behavior

- a time-aware, liquid-glass visual system

Version 2 must preserve this capability while reducing architectural fragility and making the experience consistent across screen sizes.

4. Version 2 outcomes

Version 2 is successful when:

- the same account can use PNP across web, iPhone, and Android
- data is durable, synchronized, exportable, and usable offline
- navigation and layouts remain coherent on desktop and mobile
- D.E.E.D.S. refreshes from current PNP data whenever it opens
- D.E.E.D.S. produces useful next-step guidance without requiring a paid inference service
- goals are separate from tasks but can be advanced by linked tasks and habits
- health data can come from Oura, Apple Health, and Android Health Connect with clear source labels
- reports explain meaningful trends across work, health, relationships, goals, and journal data
- the mobile applications provide enough native value to be more than wrapped websites
- privacy, consent, account deletion, and health-data handling meet store requirements

5. Product architecture

5.1 Modular product domains

The current application should be separated into maintainable modules:

- command center and navigation
- tasks, lists, schedules, and recurrence
- goals and goal progress
- health and health sources
- relationships
- business
- calendar
- mail
- journal
- reports and feedback
- D.E.E.D.S.
- identity, settings, backup, and connections

Each domain owns its components, data types, validation, and tests. Shared visual components live in a reusable design system.

5.2 Data foundation

Version 2 should use:

- an offline-capable local database on each device
- authenticated cloud storage as the synchronization authority
- versioned schemas and automatic data migrations
- append-only completion history so cleared tasks still count in reports
- conflict detection for edits made on multiple devices
- explicit source labels for imported data

- reliable export and account deletion

The existing browser data must migrate without requiring the user to rebuild the system.

5.3 Integration adapters

Oura, Google, Apple Health, and Health Connect should use separate adapters. A failed provider should not break the rest of PNP. Each connection must report:

- connected account or device
- last successful sync
- data types granted
- stale or failed data
- disconnect and resync controls

6. D.E.E.D.S. Version 2

D.E.E.D.S. becomes the decision and guidance layer across PNP.

Detect

Collect relevant, permissioned signals from PNP:

- due and overdue tasks
- task priority and estimated effort
- recurring maintenance
- calendar commitments and free windows
- important mail
- goals and recent forward motion
- sleep, energy, mood, stress, food, hydration, medication, and exercise
- time of day and expected weekly rhythm
- user preferences, constraints, and protected time

Every signal should include a source, timestamp, confidence, and staleness state.

Explore

Help the user clarify the situation without filling the screen with questions. A single Refine List action can ask only what is needed:

- What must happen today?
- What capacity do you have?
- What are you avoiding or uncertain about?

Enable

Turn uncertainty into usable choices:

- suggest a short, realistic next list
- break a task into smaller steps
- identify information or preparation needed
- find an available calendar window
- open the exact task, goal, health entry, message, or event involved

Drive

Help the user begin and continue:

- start the selected task
- schedule it
- create a preparation checklist
- draft a response
- set an internal reminder
- update the linked goal when work is completed

Sustain

Protect progress over time:

- notice repeated friction
- recognize completion and recovery
- recommend reducing load when capacity is low
- surface useful patterns in reports
- learn from accepted, dismissed, delayed, and completed suggestions

Recommendation order

D.E.E.D.S. should rank suggestions in this order:

1. immediate safety or health needs
2. fixed commitments and time-sensitive interviews
3. urgent or overdue obligations with real consequences
4. high-priority work that advances an active goal
5. recurring maintenance needed today
6. relationship and family commitments
7. recovery, discovery, and optional opportunities

The ranking must account for available time, effort, energy, dependency, user-defined priority, protected time, and the reason an item matters.

7. No-required-cost intelligence

Version 2 will separate decision intelligence from language generation.

7.1 Deterministic assistant core

The core works on every supported device and requires no language model. It handles:

- recommendation ranking
- deadline and calendar reasoning
- task-to-goal relationships
- recurring-item logic
- capacity and time budgeting
- source-aware health and emotional signals
- report calculations
- transparent explanations

This core remains authoritative. A language model may improve wording and interaction, but it cannot silently change data or override product rules.

7.2 On-device language layer

When supported by the device:

- Apple platforms can use Apple Foundation Models
- Android can use ML Kit GenAI APIs powered by Gemini Nano
- the browser may offer WebLLM as an experimental, opt-in option

These capabilities can support:

- natural-language capture
- task decomposition
- briefing and report summaries
- journal reflection prompts
- drafting
- conversational refinement of recommendations

7.3 Capability tiers

Tier A: Structured core. Available everywhere, offline, with no inference cost.

Tier B: Native on-device model. Available on supported Apple or Android devices, subject to system availability.

Tier C: Optional browser model. User-initiated download on capable browsers, with clear storage and performance expectations.

Tier D: Future optional provider. A user-supplied or paid hosted model may be considered later, but it is not required for Version 2.

7.4 Boundaries

Language models must not independently:

- diagnose medical or mental-health conditions
- recommend medication changes
- characterize another person's motives
- assign relationship blame or relationship scores
- send mail, delete data, or alter calendars without clear user confirmation
- invent facts that are not present in PNP or a connected source

Generated content must be labeled, reviewable, and backed by visible source data when it contains a recommendation.

8. Mobile application strategy

Version 2 will use Capacitor to share the existing web product across iOS and Android while adding native capabilities through platform plugins and native code.

The goal is one product with shared behavior, not three drifting implementations.

Native capabilities

- push and local notifications
- biometric unlock
- secure credential storage
- share sheet and quick capture
- deep links
- background synchronization
- haptics
- offline database
- widgets and quick actions
- native calendar presentation where it materially improves the experience
- Apple Health and Android Health Connect access

The mobile shell must provide meaningful native utility and must not simply display the website.

9. Health platform strategy

Apple

Use HealthKit only after the user grants explicit, data-type-specific permission. Imported data should be labeled by source and should remain editable only when the source permits it.

Android

Use Health Connect with equivalent permission, source, and disconnection behavior.

Precedence and averages

- user-entered data remains visible even when a connected source exists
- automatic data shows its source and last sync
- a user can override an automatically populated daily value without deleting the source record
- averages begin with the user's first day of PNP use, not earlier provider history
- missing data is missing, not zero

10. Reports and feedback

Reporting should turn collected data into useful observations.

Report layers

Daily brief: what happened, what remains, and what deserves attention.

Weekly review: progress, load, maintenance, goal movement, recovery, and notable patterns.

Monthly report: trends across tasks, goals, health, relationships, business, calendar, journal, and integrations.

Reporting rules

- distinguish correlation from causation
- show the time window and data completeness
- identify the source of imported data

- preserve completion events after tasks are cleared
- allow the user to open the underlying records
- avoid moral or diagnostic language
- explain changes in plain language
- compare like days where useful, such as Sundays with Sundays

11. Privacy, safety, and trust

Version 2 requires:

- a clear privacy policy
- informed consent for each connection
- least-privilege scopes
- encryption in transit and at rest
- secure storage for tokens and credentials
- audit history for sensitive synchronization actions
- data export
- account and cloud-data deletion
- disconnect controls
- retention settings
- no sale or advertising use of health data
- a documented process for security incidents

The product should not hardcode personal names, relationship assumptions, health conditions, or starter content into the general user experience. Personalization comes from account data, user-created content, and authorized sources.

12. Delivery roadmap

Phase 0: Scope and baseline

- freeze the Version 2 scope
- document the Version 1 data model and current integrations
- establish performance, reliability, accessibility, and privacy baselines
- create migration fixtures from real Version 1 exports

Phase 1: Stabilize and modularize

- divide the large application surface into product domains
- build shared visual, overlay, form, and list components
- standardize autosave, navigation return, overlay stacking, drag-and-drop, and mobile behavior
- add tests for task movement, recurrence, completion history, and calendar overlays

Phase 2: Identity, cloud, and offline sync

- add first-party accounts and authentication
- implement the local database
- implement encrypted cloud synchronization
- add schema migrations and conflict handling
- migrate existing local data safely

Phase 3: D.E.E.D.S. core

- define the signal and recommendation model
- implement the deterministic ranking engine
- make recommendations open their underlying records
- add user feedback and explanation
- test different day rhythms and capacity levels

Phase 4: On-device language

- add a capability detector
- integrate Apple Foundation Models
- integrate Android ML Kit GenAI
- prototype optional WebLLM
- implement structured fallbacks
- evaluate quality, latency, battery use, and privacy

Phase 5: iOS alpha

- create the Capacitor iOS application
- add native notifications, secure storage, biometrics, deep links, and background sync
- complete phone layout and accessibility testing
- distribute through TestFlight

Phase 6: Android alpha

- create the Capacitor Android application
- add equivalent native capabilities
- test across supported screen sizes and OS versions
- distribute through internal testing

Phase 7: Health integrations

- implement HealthKit
- implement Health Connect
- add permission and source management
- validate averages, overrides, and deletion

Phase 8: Beta and store readiness

- complete privacy and store disclosures
- add in-app account deletion
- perform security, accessibility, and recovery testing
- run TestFlight and Google Play beta programs
- resolve review issues before public launch

13. Version 2 committed scope

- modular application architecture
- first-party account and cross-device synchronization

- reliable offline use
- durable migration from Version 1
- goal model separate from task model
- D.E.E.D.S. deterministic assistant core
- on-device language capabilities where supported
- Capacitor iOS and Android applications
- native notifications and secure storage
- HealthKit and Health Connect
- source-aware reporting
- privacy controls, export, and account deletion

14. Deferred beyond Version 2

- a required paid hosted AI service
- autonomous external actions without confirmation
- clinical diagnosis or treatment guidance
- relationship grading or surveillance
- organizational or team administration
- a public marketplace for extensions
- broad social networking features

15. Principal risks and responses

Architecture risk

Risk: New features continue to accumulate in a single interface and become fragile.

Response: Complete modularization and tests before major mobile expansion.

Synchronization risk

Risk: Conflicts or migrations cause data loss.

Response: Use append-only history, versioned migrations, backups, fixtures, and recovery tests.

Device AI fragmentation

Risk: On-device models differ by platform and device.

Response: Make the deterministic core complete, detect capabilities, and treat language generation as enhancement.

Health privacy risk

Risk: Sensitive information is over-collected or mishandled.

Response: Use granular consent, minimum scopes, source visibility, secure storage, deletion, and privacy review.

Store review risk

Risk: A hybrid mobile app is judged to be a repackaged website.

Response: Deliver real offline use, notifications, health integrations, secure storage, widgets or quick actions, and a phone-native experience.

Assistant trust risk

Risk: D.E.E.D.S. becomes noisy, opaque, or overconfident.

Response: Limit the number of suggestions, show reasons, expose sources, learn from feedback, and keep consequential decisions with the user.

16. Definition of done

Version 2 is done when:

- a new user can create an account and understand the product without personal starter assumptions
- an existing Version 1 user can migrate without losing tasks, health entries, completion history, goals, journal entries, or settings
- web, iPhone, and Android stay synchronized and remain useful offline
- common task and check-in changes autosave
- D.E.E.D.S. refreshes on open and responds to current PNP changes
- every recommendation can explain why it was presented
- unsupported AI devices receive a complete structured experience
- imported health values show source and freshness
- reports open the underlying data
- notification, calendar, mail, Oura, Apple Health, and Health Connect failures are recoverable
- accessibility, privacy, security, and store-readiness reviews pass
- crash-free and sync-success targets are met during beta

17. Immediate Version 2 sprint

1. Inventory the Version 1 data model and create migration test files.
2. Divide the current application into domain modules.
3. Define shared types for signals, recommendations, explanations, goals, and completion events.
4. Build the deterministic D.E.E.D.S. ranking prototype.
5. Create a Capacitor proof of concept on one iPhone and one Android device.
6. Select the account, cloud database, local database, and synchronization approach.
7. Write the privacy data map before adding new collection.
8. Establish the Version 2 backlog from this plan.

18. Reference foundations

- Capacitor documentation: https://capacitorjs.com/docs
- Apple Foundation Models: https://developer.apple.com/documentation/FoundationModels
- Google ML Kit GenAI: https://developers.google.com/ml-kit/genai
- Apple App Review Guidelines: https://developer.apple.com/app-store/review/guidelines/
- Google Play health policy: https://support.google.com/googleplay/android-developer/answer/16 679 511
- WebLLM: https://github.com/mlc-ai/web-llm